MATH 7207 – Algorithms for Optimization
Mini-project Proposal Form

Your name: Clayton Schmidter

Project description.

I would like to implement 2D pose-graph SLAM. A robot moves in the plane and records noisy measurements of the relative pose between pairs of positions (e.g., LiDAR or other sensing).

This is a very standard pose-graph problem which has been solved via Gauss Newton or LM in solvers such as g2o or GTSAM. I will implement that algorithm. I will then show it in two ways: one will be using a naive start (such as dead reckoning), and the other will be using spectral methods to create a “smart” starting position.

This will demonstrate that the naive initial guess can cause these local solvers to fail on “hard” graphs and that a good guess is crucial here.

This is directly related to my research that I currently do at NU. The PIs of my current research are David Rosen (who developed SE-Sync and is huge in the PGO/SLAM space) and Frank Dellaert at Georgia Tech (who developed GTSAM). My current research exists solely in spectral initialization and I have yet to spend time to learn the optimization behind the standard PGO solvers. This project will allow for that.

Inputs and outputs

The inputs are numeric and synthetic. I will generate them, so nothing comes from a third-party dataset.

Each pose is an (x, y, heading). The graph is a list of edges. An edge says which two poses it connects, the measured relative pose between them, and a positive weight for how much that measurement should be trusted. Most edges are consecutive odometry. A few are loop closures. The measurements are the true relative pose plus Gaussian noise.

The outputs are the estimated poses, the final cost, and the iteration count, once from the dead-reckoning start and once from the spectral start. I will also plot the true path, the drifted dead-reckoning path, and both optimized paths, plus a plot of cost versus iteration for the two starts. Pose error is measured against the ground truth after fixing the global gauge, since shifting or rotating the whole map does not change the cost.

However, if needed I may also run one small public g2o benchmark as an extra figure. The tests use the synthetic graphs, where the true poses are known.

Solution algorithm.

Please describe the algorithm you will use to solve the problem. In best case, this should be a numbered list of steps, similar to what is presented in class lectures.

1. Build the measurement graph from the edges.

2. Make two starting guesses. Dead reckoning chains the odometry edges in order and ignores the loop closures. The spectral start builds the connection Laplacian, takes the eigenvectors for the smallest eigenvalues, and projects each block onto SE(2).

3. The cost is the maximum-likelihood pose-graph cost. For each edge from pose i to pose j, with measurement Z_{ij} and weight κ_{ij}, add κ_{ij} times the squared length of Log(Z_{ij}^{-1} X_i^{-1} X_j). Log turns a relative pose into a 3-vector (x, y, heading).

4. Refine with Levenberg–Marquardt. Each iteration builds the residual r and the Jacobian J, then solves (J^T J + lambda I) delta = -J^T r and updates the poses with the exponential map. If the cost goes down, accept the step and shrink lambda. If not, reject it and grow lambda.

5. Fix the first pose at the identity so the whole map cannot slide or spin. Stop when the gradient is small enough, or when the iteration limit is hit.

6. Run steps 4 and 5 from both starts and compare the cost, the iteration count, and the pose error.

Corner cases

There are a few corner cases:

Gauge. Adding the same shift and rotation to every pose does not change the cost. The first pose is fixed at the identity.

Disconnected graph. Only the component tied to the first pose is solvable. The program should stop and say so, instead of returning a meaningless answer.

A very noisy or sparsely connected graph. The spectral start can still be wrong, and refinement then stops at a local minimum. That run is a failure against the ground truth,

even if the gradient is about zero.

This is not an exhaustive list of corner cases.

Testing

I will test multiple scenarios:

Noiseless connected graph. Both starts should recover the true poses, up to the gauge, with cost near zero and pose error under 1e-5.

A toy graph with three poses and three edges. Check the residual and the Jacobian against a finite difference before trusting Levenberg–Marquardt.

A harder graph with moderate noise and one loop closure. The spectral start should finish near the ground truth. Dead reckoning should either get there too or finish at a higher cost with a visibly wrong path. Both are checked against the true poses.

A disconnected graph. This should return a clear failure.

This is not an exhaustive list of test cases.